

LAPORAN PRAKTIKUM PBO 2

disusun Oleh:

Muhammad Faiz An Anri

NIM 2511532021

Dosen Pengampu: Nurfiah S.ST, M.KOM

Asisten Pratikum: Siti Fadhilah Rahmi



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

Kata Pengantar

Ucapan syukur kita lafalkan kepada Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah PBO.

Ucapan terima kasih disampaikan kepada Bapak Nurfiah, M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran konsep perulangan dalam bahasa pemrograman Java.

Daftar Isi

Kata Pengantar 2

Daftar Isi 3

BAB I..... 5

BAB II..... 7

BAB III 10

BAB IV 15

BAB I

Pendahuluan

1. Latar Belakang

Pemrograman Berorientasi Objek (PBO) memungkinkan suatu sistem direpresentasikan melalui objek-objek yang saling berhubungan. Pada praktikum ini, konsep tersebut diterapkan melalui relasi antarobjek menggunakan Class Diagram dan Asosiasi one-to-many. Sistem perbankan yang sebelumnya hanya melakukan transaksi setor dan tarik saldo dikembangkan agar setiap aktivitas transaksi dapat direpresentasikan sebagai objek Transaksi dan disimpan di dalam objek Rekening.

Penerapan relasi tersebut menggunakan struktur data dinamis ArrayList untuk mengelola kumpulan objek transaksi yang dimiliki oleh sebuah rekening. Dengan demikian, sistem tidak hanya dapat melakukan transaksi, tetapi juga menyimpan dan menampilkan riwayat transaksi yang telah dilakukan.

2. Tujuan

- Memahami dan menerjemahkan Class Diagram ke dalam implementasi program menggunakan bahasa Java.
- Memahami serta menerapkan konsep relasi Asosiasi one-to-many antarobjek.
- Mengimplementasikan pengelolaan kumpulan objek menggunakan ArrayList serta menerapkan pencatatan dan penampilan riwayat transaksi.

3. Manfaat

- Meningkatkan pemahaman mengenai hubungan antarclass dan objek dalam Pemrograman Berorientasi Objek.
- Melatih penggunaan ArrayList untuk menyimpan dan mengelola kumpulan objek secara dinamis.
- Memahami penerapan pencatatan riwayat transaksi sehingga aktivitas setor dan tarik tunai dapat direkam serta ditampilkan kembali kepada pengguna.

BAB II

Isi

1. Membuat Langkah transaksi

```
package PraktikumMinggu2;
```

```
public class Transaksi {  
    String idTransaksi;  
    String jenis;  
    double nominal;  
  
    public Transaksi(String id, String jenis, double nom) {  
        this.idTransaksi = id;  
        this.jenis = jenis;  
        this.nominal = nom;  
    }  
  
    public void cetakDetail() {  
        System.out.println("ID: " + idTransaksi + " | Jenis: " +  
            jenis + " | Nominal : Rp" + nominal);  
    }  
}
```

Class ini mengikuti spesifikasi modul bahwa Transaksi mempunyai tiga atribut, yaitu idTransaksi, jenis, dan nominal, serta memiliki method cetakDetail() untuk menampilkan detail transaksi.

2. Membuat Relasi pada Rekening.java

```
public class Rekening {  
    ArrayList<Transaksi> riwayatTransaksi;  
    public Rekening(String norek, String nama, double saldo) {  
        this.nomorRekening = norek;  
        this.namaPemilik = nama;  
        this.saldo = saldo;  
        this.riwayatTransaksi = new ArrayList<>();  
        System.out.println("Rekening atas nama : " + namaPemilik + " Berhasil  
dibuat.");  
    }  
}
```

Class ini mengikuti spesifikasi modul bahwa Transaksi mempunyai tiga atribut, yaitu idTransaksi, jenis, dan nominal, serta memiliki method cetakDetail() untuk menampilkan detail transaksi.

3. Mencatat Riwayat Transaksi

```

public void setorTunai(double nominal) {
    if (nominal > 0) {
        if (nominal > Tsetor) {
            Tsetor = nominal;
        }
        else if (nominal < Lsetor) {
            Lsetor = nominal;
        }
        saldo += nominal;
        String idTrx = "TRX-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp" + nominal + " Berhasil. Saldo
saat ini: Rp" + saldo);
    }
    else {
        System.out.println("Gagal: Nominal setor harus lebih dari 0!");
    }
}

```

Intinya, objek Transaksi hanya dimasukkan ke riwayatTransaksi setelah transaksi berhasil, sesuai instruksi modul. Untuk penarikan, modul secara khusus menentukan jenis transaksi sebagai "Debit" dan menggunakan prefiks ID "TRX-T-".

BAB III

Latihan

1. Integrasi Rekam Jejak Penarikan (30%)

```

public void tarikTunai(double nominal) {

    if (nominal < 10000) {

        System.out.println("Transaksi Gagal : Minimal nominal penarikan 10.000");

    }

    else if (nominal > saldo) {

        System.out.println("Transaksi Gagal : Saldo tidak mencukupi. Saldo Anda:
Rp" + saldo);

    }

    else {

        if (nominal > Ttarik) {

            Ttarik = nominal;

        }

        else if (nominal < Lta) {

            Lta = nominal;

        }

        saldo -= nominal;

        String idTrx = "TRX-T-" + System.currentTimeMillis();

        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);

        riwayatTransaksi.add(trxBaru);

        System.out.println("Tarik tunai Rp" + nominal +

            " Berhasil, Saldo saat ini: Rp" + saldo);

        tAR += nominal;

    }

}

```

Kode tersebut merupakan method tarikTunai() yang menangani proses penarikan uang sekaligus mencatat transaksi dan beberapa statistik penarikan. Pertama, program melakukan validasi nominal. Jika nominal < 10000 , transaksi ditolak karena batas minimum penarikan adalah Rp10.000. Jika nominal lebih besar dari saldo, transaksi juga ditolak karena saldo tidak mencukupi. Jika kedua kondisi tersebut tidak terpenuhi, proses penarikan dilanjutkan. Pada bagian else, Ttarik digunakan untuk menyimpan nilai penarikan terbesar, sedangkan Lta digunakan untuk menyimpan nilai penarikan terkecil. Setelah itu saldo dikurangi dengan saldo $-=$ nominal. Program kemudian membuat ID transaksi menggunakan waktu saat ini, misalnya "TRX-T...", lalu membuat objek Transaksi dan memasukkannya ke riwayatTransaksi. Terakhir, program menampilkan pesan bahwa penarikan berhasil dan menambahkan nominal penarikan ke tAR melalui tAR $+=$ nominal. Jadi, secara keseluruhan method ini melakukan validasi penarikan $\rightarrow$ memperbarui statistik $\rightarrow$ mengurangi saldo $\rightarrow$ mencatat riwayat transaksi $\rightarrow$ menampilkan hasil transaksi.

2. Fitur Cetak Mutasi Rekening

Fitur mutasi rekening tidak merupakan bagian dari quiz yang dilakukan kemarin, jadi untuk laporan saya akan hanya menggunakan kode yang telah saya buat

```

public void CetakMutasi() {
    int len = riwayatTransaksi.size() - 1;

    if (len != 0) {
        if (len >= 3) {
            for (int i = len; i > len - 3; i--) {
                riwayatTransaksi.get(i).cetakDetail();
            }
        }
        else {
            for (int i = 0; i <= len; i++) {
                riwayatTransaksi.get(i).cetakDetail();
            }
        }
    }
    else {
        System.out.println("Tidak ada transaksi yang dilakukan");
    }
}

```

Method CetakMutasi() digunakan untuk menampilkan riwayat transaksi pada rekening dengan mengambil data dari riwayatTransaksi. Variabel len digunakan untuk mendapatkan indeks transaksi terakhir, kemudian jika terdapat minimal tiga transaksi, program menampilkan tiga transaksi terakhir dari yang paling baru menggunakan perulangan mundur dan method cetakDetail(). Jika jumlah transaksi kurang dari tiga,

seluruh transaksi yang tersedia akan ditampilkan. Namun, kondisi `if (len != 0)` memiliki kekurangan karena satu transaksi memiliki nilai `len = 0` dan akan dianggap tidak ada transaksi; kondisi yang lebih tepat adalah memeriksa apakah riwayatTransaksi kosong.

BAB IV

Kesimpulan

1. Kesimpulan

Pada praktikum ini diajarkan cara membuat relasi antar kelas di dalam program java (Transaksi class dan Rekening class). Kemudian juga diajarkan cara untuk mengelolah objek-objek yang dimiliki.

Jadi yang dipelajari adalah interaksi objek dan cara mengendalikan relasi objek-objek tersebut.